```
/*
 * File:   rtc_master.c
 * Author: sagar
 *
 * Created on 23 July, 2025, 10:01 AM
 */

// CONFIG
#pragma config FOSC = HS        // Oscillator Selection bits (HS oscillator)
#pragma config WDTE = OFF       // Watchdog Timer Enable bit (WDT disabled)
#pragma config PWRTE = OFF      // Power-up Timer Enable bit (PWRT disabled)
#pragma config BOREN = OFF      // Brown-out Reset Enable bit (BOR disabled)
#pragma config LVP = OFF        // Low-Voltage (Single-Supply) In-Circuit Serial Programming Enable bit (RB3 is digital I/O, HV on MCLR must be used for programming)
#pragma config CPD = OFF        // Data EEPROM Memory Code Protection bit (Data EEPROM code protection off)
#pragma config WRT = OFF        // Flash Program Memory Write Enable bits (Write protection off; all program memory may be written to by EECON control)
#pragma config CP = OFF         // Flash Program Memory Code Protection bit (Code protection off)


#include <xc.h>
#include <stdint.h>

#define _XTAL_FREQ 20000000
#define RS RC2
#define EN RC1
```

```
void Lcdinit(void);
void i2cInit(const unsigned long baud);
void i2cWrite(unsigned char data);
int i2cRead(int ack);
void i2cWait(void);
void i2cStop(void);
void i2cStart(void);
void i2cRestart(void);
void LcdData(uint8_t i);
void LcdCommand(uint8_t i);

int bcdToDec(int);
int decToBcd(int);

void setTime(void);
void update(void);

char msg1[7] = "TIME: ";
char msg2[7] = "DATE: ";

int sec, min, hour, date, month, year;
char ampm[3] = "AM";  // To store AM/PM string

void main(void)
{
   Lcdinit();
   LcdCommand(0x80);
   for(int i = 0; i <= 5; i++) LcdData(msg1[i]);
```

```
LcdCommand(0xC0);
for(int i = 0; i <= 5; i++) LcdData(msg2[i]);

i2cInit(100);  // I2C at 100kHz

setTime(); // Run only once, then comment after flashing

__delay_ms(500);
while(1)
{
   update();

   LcdCommand(0x85); // TIME display position
   LcdData((hour / 10) + '0');
   LcdData((hour % 10) + '0');
   LcdData(':');
   LcdData((min / 10) + '0');
   LcdData((min % 10) + '0');
   LcdData(':');
   LcdData((sec / 10) + '0');
   LcdData((sec % 10) + '0');
   LcdData(' ');
   LcdData(ampm[0]);
   LcdData(ampm[1]);

   LcdCommand(0xC5); // DATE display position
   LcdData((date / 10) + '0');
```

```
        LcdData((date % 10) + '0');
        LcdData('/');
        LcdData((month / 10) + '0');
        LcdData((month % 10) + '0');
        LcdData('/');
        LcdData((year / 10) + '0');
        LcdData((year % 10) + '0');

        __delay_ms(1000);
    }
}

// ---------- I2C Functions ----------
void i2cInit(const unsigned long baud)
{
    TRISC3 = 1; // SCL
    TRISC4 = 1; // SDA

    SSPCON = 0x28;
    SSPCON2 = 0x00;
    SSPSTAT = 0x00;
    SSPADD = (_XTAL_FREQ / (4 * baud * 100)) - 1;
}

void i2cWait(void)
{
    while ((SSPSTAT & 0x04) || (SSPCON2 & 0x1F));
}
```

```
void i2cStart(void)   { i2cWait(); SEN = 1; }
void i2cStop(void)    { i2cWait(); PEN = 1; }
void i2cRestart(void) { i2cWait(); RSEN = 1; }

void i2cWrite(unsigned char data)
{
   i2cWait();
   SSPBUF = data;
}

int i2cRead(int ack)
{
   int value;
   i2cWait(); RCEN = 1;
   i2cWait(); value = SSPBUF;
   i2cWait(); ACKDT = (ack) ? 0 : 1;
   ACKEN = 1;
   return value;
}

// ---------- RTC Set Function (12-hour + PM) ----------
void setTime()
{
   i2cStart();
   i2cWrite(0xD0); // RTC Address + write
   i2cWrite(0x00); // Start at register 0
```

```
  i2cWrite(decToBcd(30));     // Seconds = 30
  i2cWrite(decToBcd(59));     // Minutes = 59

  // Set hour = 11 | bit6 = 1 (12-hr) | bit5 = 1 (PM)
  uint8_t hourReg = decToBcd(11); // 11
  hourReg |= (1 << 6); // 12-hour mode
  hourReg |= (1 << 5); // PM bit
  i2cWrite(hourReg);   // Hours = 11 PM (12-hour mode)

  i2cWrite(0x02);              // Day of week (optional)
  i2cWrite(decToBcd(21));      // Date = 21
  i2cWrite(decToBcd(7));       // Month = July
  i2cWrite(decToBcd(25));      // Year = 2025

  i2cStop();
}

// ---------- RTC Read and Format ----------
void update(void)
{
  i2cStart();
  i2cWrite(0xD0);
  i2cWrite(0x00);
  i2cRestart();
  i2cWrite(0xD1);

  sec = bcdToDec(i2cRead(1));
  min = bcdToDec(i2cRead(1));
```

```
  uint8_t rawHour = i2cRead(1);
  if (rawHour & (1 << 6))  // 12-hour format
  {
    hour = bcdToDec(rawHour & 0x1F);  // bits 0?4 are hour in BCD
    if (rawHour & (1 << 5))          // PM bit
      ampm[0] = 'P';
    else
      ampm[0] = 'A';
  }
  else
  {
    hour = bcdToDec(rawHour & 0x3F);  // 24-hour fallback
    ampm[0] = ' ';
    ampm[1] = ' ';
  }
  ampm[1] = 'M';

  i2cRead(1); // Skip day
  date  = bcdToDec(i2cRead(1));
  month = bcdToDec(i2cRead(1));
  year  = bcdToDec(i2cRead(0)); // Last read with NACK

  i2cStop();
}

// ---------- BCD Conversions ----------
int bcdToDec(int val) {
```

```
    return ((val >> 4) * 10 + (val & 0x0F));
}
int decToBcd(int val) {
    return (((val / 10) << 4) | (val % 10));
}

void Lcdinit(void) {
    TRISC = 0x00;  // Set PORTC as output (control signals)
    TRISD = 0x00;  // Set PORTD as output (data signals)

    __delay_ms(100);

    LcdCommand(0x30);
    __delay_ms(5);
    LcdCommand(0x30);
    __delay_ms(5);
    LcdCommand(0x30);
    __delay_ms(5);
    LcdCommand(0x38);
    __delay_ms(5);
    LcdCommand(0x0C);
    __delay_ms(5);
    LcdCommand(0x01);
    __delay_ms(5);
}

void LcdData(uint8_t i) {
    RS = 1;
```

```
    PORTD = i;
    EN = 1;
    __delay_ms(2);
    EN = 0;
}

void LcdCommand(uint8_t i) {
    RS = 0;
    PORTD = i;
    EN = 1;
    __delay_ms(2);
    EN = 0;
}
```